

Golden Record Curator

Guida al codice — come tutto si intreccia

Documento di accompagnamento al progetto

EPICODE Institute of Technology — Final Project Assignment

33 classi · 218 contenuti reali · 4 pattern obbligatori + 2 opzionali

Compilato e verificato: nessun warning, nessun crash, nessuno stack trace

Indice

#	Contenuto	Pag.
1	Che cosa hai in mano — il progetto in trenta righe, i 33 file raggruppati per mestiere	2
2	Come tutto si intreccia — il viaggio di 'load' file per file, dove il Composite lavora, cosa succede quando qualcosa va storto	5
3	Dove sta ogni requisito — i 4 pattern obbligatori, i 2 opzionali, le 5 tecnologie, le 6 penalita' chiuse	8
4	I quattro pilastri, riga per riga — astrazione, incapsulamento, ereditarieta', polimorfismo, e composizione contro ereditarieta'	11
5	Le dieci domande, con le risposte — da leggere ad alta voce	12
6	La demo, minuto per minuto — sette comandi in ordine, le tre regole del colloquio, il conto dei punti	14

Capitolo 1 — Che cosa hai in mano

Il progetto in trenta righe, e la mappa dei file.

1.1 · Che cos'è, fisicamente

È un'applicazione Java SE **da terminale**. Non ha finestre, non ha browser, non ha database, non ha rete. La lanci, scrivi un comando, lei stampa testo e aspetta il comando successivo.

Riceve **un file**: data/voyager-1977.csv, 250 righe. Lo legge, ne costruisce in memoria un albero di oggetti Java, e ti lascia interrogare quell'albero.

Numero	Che cos'è
32	sezioni: contenitori (Lato 1, Saluti, Lingue antiche...)
218	contenuti reali: 115 immagini, 55 saluti, 21 suoni, 27 brani
6	livelli di profondità dell'albero
6903	secondi totali, contro i 6600 di capienza: fuori budget
87:16	durata della musica — combacia col dato pubblicato dalla NASA

1.2 · I 33 file, raggruppati per lavoro

```

it/zagomattia/epicode/goldenrecord/
|
|-- App.java ..... IL PUNTO DI PARTENZA: monta gli oggetti e avvia
|
|-- content/ ..... IL DOMINIO. Qui vive il Composite.
|   |-- Content.java ..... l'interfaccia comune (Component)
|   |-- ContentType.java ..... enum con comportamento
|   |-- Section.java ..... il contenitore (Composite)
|   |-- Disc.java ..... la radice, aggiunge i vincoli fisici
|   |-- leaves/
|       |-- AbstractContent.java base comune delle foglie
|       |-- Greeting.java ..... i 55 saluti
|       |-- MusicTrack.java ..... i 27 brani (+ BUILDER)
|       |-- EarthSound.java ..... i 21 suoni
|       |-- EncodedImage.java ... le 115 immagini
|
|-- factory/ContentFactory.java FACTORY: riga CSV -> oggetto giusto
|
|-- iterator/ ..... ITERATOR: due modi di percorrere l'albero
|   |-- PlaybackIterator.java .. ordine di incisione, contenitori inclusi
|   |-- EngravableIterator.java solo le foglie
|   |-- EngravableContents.java adattatore per il for-each
|
|-- manifest/ ..... LETTURA DEI DATI
|   |-- ManifestRecord.java .... una riga grezza (DTO)
|   |-- PathSanitizer.java ..... difesa dal path traversal
|   |-- DiscLoader.java ..... righe piatte -> albero
|   |-- provider/ManifestProvider.java il contratto
|   |-- spi/CsvManifestProvider.java l'implementazione CSV
|   |-- spi/CsvReader.java ..... splitter di una riga
|
|-- exceptions/ ..... EXCEPTION SHIELDING
|   |-- CuratorException.java .. la base, con DUE messaggi
|   |-- (8 eccezioni concrete) . una per ogni guasto previsto
|   |-- SafeExecutor.java ..... IL CONFINE: unico punto di cattura
|   |-- Result.java ..... contenitore generico dell'esito
|   |-- CuratorAction.java ..... interfaccia funzionale nostra
|   |-- TraceId.java ..... il codice di correlazione
|
|-- logging/ ..... I DUE CANALI
|   |-- CuratorLogger.java ..... configura file + console
|   |-- SafeConsoleFormatter.java formatter che NON stampa lo stack trace

```

```
|
|-- validation/ ..... LE TRE REGOLE DI INCIDIBILITA'
|   |-- DiscValidator.java ..... formato, capienza, copertura
|   `-- ValidationIssue.java ... un singolo rilievo
|
|-- curation/ ..... STRATEGY: che cosa sacrificare
|   |-- CurationPolicy.java .... il contratto
|   |-- MaximizeLanguageCoverage.java  protegge le lingue
|   `-- ShortestFirstPolicy.java ..... massimizza il numero di pezzi
|
`-- cli/ ..... L'INTERFACCIA UTENTE
    |-- CuratorConsole.java .... il ciclo dei comandi
    |-- ConsoleWriter.java ..... UNICO punto che scrive a video
    `-- InputSanitizer.java .... ripulisce quello che scrivi
```

La regola per orientarti

Ogni cartella ha **un solo mestiere**. content contiene, iterator naviga, factory costruisce, manifest legge, validation controlla, curation decide, exceptions protegge, cli parla con te. Se il professore ti chiede dove sta una cosa, la risposta è sempre il nome del mestiere.

Capitolo 2 — Come tutto si intreccia

Il viaggio di un comando, file per file. Questa è la domanda numero due dell'esame.

2.1 · Scrivi `load` e premi Invio

Segui la catena. Ogni riga è un file diverso, e il passaggio da uno all'altro è sempre una chiamata di metodo.

```

1. CuratorConsole.run()
   legge la riga con Scanner

2. SafeExecutor.execute("COMMAND", () -> dispatch(riga))
   |
   | DA QUI IN POI SIAMO PROTETTI: qualunque cosa esploda,
   | esplode dentro questo try
   v
3. InputSanitizer.sanitizeCommand(riga)
   toglie spazi, limita la lunghezza, elimina caratteri di controllo

4. CuratorConsole.dispatch()
   spezza in verbo + argomento, riconosce "load"

5. InputSanitizer.sanitizeFileName("voyager-1977.csv")
   whitelist: niente ".."

6. DiscLoader.load(nome, 6600, 3300)
   |
+--> 7. CsvManifestProvider.loadRecords(nome)
   |
   | +--> 8. PathSanitizer.sanitize(nome)
   |         resolve -> normalize -> startsWith(base)
   |         se esce dalla cartella: InsecurePathException
   |
   | +--> 9. Files.newBufferedReader(percorso, UTF_8)
   |         try-with-resources: si chiude da solo
   |
   | +--> 10. CsvReader.splitLine(riga) x 250
   |         gestisce le virgolette
   |
   |         restituisce List<ManifestRecord> (250 righe grezze)
   |
+--> 11. PASSATA 1: per ogni riga
   |     ContentFactory.createContent(riga)
   |     |
   |     +--> ContentType.parse("GREETING") -> enum, o eccezione
   |     +--> new Greeting(...) -> valida nel costruttore
   |
   |     mette tutto in una HashMap: id -> oggetto
   |
+--> 12. PASSATA 2: per ogni riga

```

```

    Section.addContent(figlio)
    |
    +-> guardAgainstCycle()    -> niente anelli

```

```

13. Result.success(true) -> CuratorConsole stampa con ConsoleWriter

```

Perché due passate e non una

Perché una riga può nominare un genitore che compare **più avanti** nel file. Con due passate l'ordine delle righe smette di contare: prima creo tutti gli oggetti, poi li attacco. È una risposta da dare senza esitare, perché il professore la chiede spesso quando vede un doppio ciclo.

2.2 · Scrivi budget: dove il Composite fa il suo lavoro

```

CuratorConsole.doBudget()
|
+--> disc.getSides()          i figli diretti del disco
|
+--> side.getPlaybackTime()    <-- QUI PARTE LA RICORSIONE
    |
    | Section.getPlaybackTime():
    |   total = 0
    |   for (Content child : children)
    |       total += child.getPlaybackTime();    <-- POLIMORFISMO
    |   return total
    |
    | Se il figlio e' una FOGLIA:
    |   AbstractContent.getPlaybackTime() -> restituisce il campo
    |   LA RICORSIONE SI FERMA
    |
    | Se il figlio e' una SEZIONE:
    |   rientra nel metodo qui sopra, un livello piu' giu'
    v
+--> ConsoleWriter.formatTime(secondi)    l'unico posto che fa mm:ss

```

Non c'è **nemmeno un** if che distingua un contenitore da una foglia. La distinzione la fa la JVM a runtime guardando la classe reale dell'oggetto. Il nome tecnico è **dynamic dispatch**, e la condizione di terminazione non è scritta nel codice: è la forma dei dati. Le foglie sono i punti in cui l'albero finisce.

2.3 · Quando qualcosa va storto

```
Scrivi: load ../../etc/passwd
```

```
PathSanitizer -> InsecurePathException
|               diagnostic: "Path '../../etc/passwd' resolves to
|                           '/etc/passwd', which is outside..."
|               userMessage: "The requested file path is not allowed."
|
|   risale SENZA essere catturata da nessuno
|   (i livelli intermedi non catturano: non possono decidere niente)
v
SafeExecutor.execute()
|
+--- catch (CuratorException) ---+
|                                   |
v                                   v
curator.log                       ConsoleWriter
WARNING + diagnostic              "[ERR-000003] The requested file
+ stack trace completo            path is not allowed."
```

La frase da dire

« Non sto nascondendo l'errore: lo sto **instradando**. Il dettaglio tecnico va nel log, dove serve allo sviluppatore. L'utente riceve una frase sicura più sei caratteri. Se mi riporta quei sei caratteri, io trovo l'evento esatto nel file di log. »

Capitolo 3 — Dove sta ogni requisito

La tabella da tenere aperta. Il professore indica una voce, tu apri il file.

3.1 · I quattro pattern obbligatori

Pattern	File	La riga da mostrare
Composite (4 pt)	content/Content.java content/Section.java content/Disc.java	Section.getPlaybackTime() — il ciclo for che somma i figli. E AbstractContent.getPlaybackTime(), che restituisce il campo: sono il caso ricorsivo e il caso base.
Factory (3 pt)	factory/ContentFactory.java	Lo switch (type): cinque rami, uno per costante dell'enum. Il ramo MUSIC chiama il Builder.
Iterator (4 pt)	iterator/PlaybackIterator.java iterator/EngravableIterator.java	Il ciclo a ritroso dentro next(): for (int i = size-1; i >= 0; i--). È la riga che si sbaglia più spesso.
Exception Shielding (5 pt)	exceptions/CuratorException.java exceptions/SafeExecutor.java logging/SafeConsoleFormatter.java	I due catch di SafeExecutor. E i due metodi astratti di CuratorException: getDiagnostic() e getUserMessage().

3.2 · I due pattern opzionali — e perché solo due

Pattern	File	Perché proprio questo
Strategy (4 pt)	curation/CurationPolicy.java + due implementazioni	Contiene la vera logica di business: quando sfori, che cosa tagli? Struttura e criterio sono due assi indipendenti.
Builder (3 pt)	content/leaves/MusicTrack.java	Sette campi, tre opzionali. E si compone con la Factory senza sovrapporsi: la Factory sceglie quale classe, il Builder come assemblarla.

« Perché non hai messo più pattern? »

« Perché un pattern è un costo, pagato in indirezione, e si paga solo quando il problema lo chiede. DiscValidator esegue tre controlli in tre metodi privati: trasformarlo in una catena di responsabilità lo avrebbe reso più lungo senza renderlo migliore. Ho preferito un progetto che so spiegare in ogni sua riga a uno più ricco che so spiegare a metà. »

3.3 · Le cinque tecnologie obbligatorie

Tecnologia	Dove, e la motivazione da dire
Collections (3 pt)	ArrayList per i figli di Section: l'ordine di incisione è il dato, quindi serve una List e non un Set; la lettura è continua e la scrittura rara. ArrayList come pila negli iteratori: add() e remove(size-1) sono entrambe O(1). HashMap come indice id -> Content in DiscLoader: O(1) invece di O(n) a ogni ricerca. TreeSet per le lingue: unicità e ordine, così il report è deterministico.
Generics (3 pt)	Result<T> in exceptions/: porta l'esito di qualunque operazione in modo type safe. Senza <T> conterrebbe un Object e ogni chiamante scriverebbe un cast. CuratorAction<T> è la nostra interfaccia funzionale.
Java I/O (3 pt)	Tre usi distinti: Files.newBufferedReader col try-with-resources in CsvManifestProvider; la lettura di disc.properties dal classpath in App; e StandardCharsets.UTF_8 esplicito, senza cui i saluti in cirillico e arabo diventano illeggibili su Windows.
Logging (2 pt)	CuratorLogger con due handler. Il file tiene tutto. La console è spenta (Level.OFF), perché i messaggi di log sono diagnostici e non devono raggiungere l'utente.
JUnit (3 pt)	Sei classi di test. La più importante è ExceptionShieldingTest: verifica che il messaggio all'utente non contenga mai un nome di classe, un package o un percorso.

3.4 · La sicurezza — le sei penalità, chiuse una per una

Penalità	Costo	Dove l'ho chiusa
Stack trace visibili	-5	SafeExecutor è l'unico punto di cattura; SafeConsoleFormatter non stampa mai il throwable; la console del logger è a Level.OFF
Crash su input non valido	-5	Il while di CuratorConsole non può essere interrotto: ogni comando passa da SafeExecutor, il cui secondo catch prende anche l'imprevisto
Valori hardcoded	-3	Capienze e soglie in disc.properties, lette da App con fallback documentati
Nessuna sanitizzazione	-3	InputSanitizer (whitelist) e PathSanitizer (resolve → normalize → startsWith)
Leak di eccezioni	-3	Ogni CuratorException ha due messaggi separati per due destinatari
Logging fatto male	-2	Due canali disgiunti, livelli corretti, trace id di correlazione

Le due verifiche da eseguire davanti a lui

```
grep -rn "printStackTrace" src/main/java → nessun risultato
```

```
grep -rn "System.out" src/main/java → solo ConsoleWriter.java
```

Tre secondi, e valgono più di qualunque spiegazione: dimostrano che la separazione dei canali è un **fatto verificabile**, non un'intenzione.

Capitolo 4 — I quattro pilastri, riga per riga

Se ti chiede « dove si vede l'incapsulamento? », non improvvisare: apri il file.

Pilastro	File e riga	Che cosa dire
Astrazione	content/Content.java	« L'interfaccia dichiara che a un contenuto si può chiedere la durata, senza dire se sia un campo o una somma su cinquanta discendenti. La prova è che <code>DiscValidator</code> calcola il totale senza nominare mai una classe concreta. »
Incapsulamento	content/Section.java <code>getChildren()</code>	« Il campo <code>children</code> è <code>private final</code> , e il getter restituisce una vista non modificabile . Attenzione: <code>final</code> protegge il riferimento, non l'oggetto. Se restituissi la lista vera, chiunque potrebbe chiamare <code>clear()</code> e svuotare il disco senza passare da <code>addContent()</code> , dove stanno i controlli. »
Ereditarietà	leaves/AbstractContent.java content/Disc.java	« Usata dove la relazione IS-A è vera. Un <code>Greeting</code> è un contenuto foglia, quindi eredita <code>id</code> , <code>titolo</code> , <code>durata</code> e la loro validazione. Un <code>Disc</code> è una <code>Section</code> , quindi riusa le tre aggregazioni ricorsive invece di riscriverle. »
Polimorfismo	content/Section.java riga <code>total += child.getPlaybackTime()</code>	« Il compilatore sa solo che <code>child</code> è un <code>Content</code> . A runtime la JVM guarda la classe reale e sceglie l'implementazione: se è una foglia restituisce un campo, se è una sezione rientra e scende. È il dynamic dispatch , ed è ciò che rende possibile il <code>Composite</code> . »

4.1 · Composizione contro ereditarietà

« **Perché** `Section` **non estende** `ArrayList`? »

« Perché la relazione IS-A sarebbe falsa: una sezione **ha** una lista, non **è** una lista. Ha un `id`, un `titolo`, una `capienza`, un `genitore`; una lista no.

E c'è una conseguenza pratica grave: erediterei anche `clear()`, `set()` e `addAll()`, che scavalcano `guardAgainstCycle()`. Regalerei a chiunque un modo di svuotare il disco o di creare un anello senza passare da nessun controllo.

Uso entrambe dove ciascuna è vera: eredito per il contratto, compongo per il contenuto. »

Capitolo 5 — Le dieci domande, con le risposte

Leggile ad alta voce. Ognuna sta in meno di trenta secondi.

1. « Parlami del progetto. Che problema risolve? »

Golden Record Curator assiste la curatela di un messaggio interstellare, prendendo come caso reale il Disco d'Oro del Voyager del 1977. Carica da un file CSV un catalogo di 218 contenuti reali e li rappresenta come un albero profondo sei livelli. Su quell'albero calcola la durata totale, verifica che rientri nei 110 minuti di capienza fisica, misura la copertura linguistica e, quando il budget è superato, propone che cosa sacrificare secondo criteri editoriali intercambiabili. Il problema è la **curatela sotto vincolo**: la capienza è finita, l'umanità no.

2. « Qual è il flusso principale? »

Il comando arriva da tastiera, viene ripulito da `InputSanitizer`, spezzato in verbo e argomento, ed eseguito **dentro** `SafeExecutor`, che è l'unico punto di cattura dell'applicazione. Il comando interroga l'albero, e il risultato viene stampato da `ConsoleWriter`, che è l'unico punto che scrive a video. Se qualcosa fallisce, il dettaglio va nel log e l'utente riceve un messaggio sicuro con un codice di correlazione.

3. « Perché Composite e non una semplice List? »

Perché la struttura annida a profondità arbitraria: Musica → Classica → Bach, sei livelli nel manifest reale. Una `List` è piatta, e mi costringerebbe a scrivere l'aggregazione ricorsiva **fuori** dal modello, in un servizio che fa `instanceof` per capire che cosa ha in mano. È l'alternativa che ho valutato e scartato, perché ogni nuovo tipo di contenuto avrebbe richiesto un ramo in più in ogni metodo di attraversamento. Col `Composite` il metodo che somma è di sei righe e non contiene un solo `if`.

4. « Come funziona la ricorsione? Come eviti i cicli infiniti? »

Non è una ricorsione scritta a mano: è **dynamic dispatch**. `Section` cicla sui figli e chiama `getPlaybackTime()`; a runtime la JVM sceglie l'implementazione in base al tipo reale, e sulle foglie la ricorsione si ferma. La condizione di terminazione è la forma dei dati. I cicli li impedisco in `guardAgainstCycle()`, chiamato da `addContent()` prima di ogni inserimento: risale la catena dei parent e rifiuta con `CyclicStructureException`. Senza quel controllo un anello produrrebbe `StackOverflowError`, cioè un crash.

5. « Perché non usi semplicemente `new`? »

Perché al momento della lettura non so quale classe istanziare: dipende dalla colonna `type`, che è un dato di runtime, mentre `new` richiede il nome della classe a compile time. In più, se spargessi `new Greeting` dentro il lettore CSV, il lettore dipenderebbe da tutte e quattro le classi concrete: un modulo di I/O accoppiato all'intero dominio.

6. « Se aggiungo un nuovo tipo di contenuto, cosa devi modificare? »

Tre cose, tutte **additive**: una costante in `ContentType`, una classe che estende `AbstractContent`, un ramo nello switch della `Factory`. Nessun metodo esistente viene riscritto: né il lettore, né gli iteratori, né il validatore, né la CLI. È l'**Open/Closed Principle**, ottenuto per costruzione.

7. « Perché Iterator e non un for-each sui figli? »

Un `for-each` sui figli visita **un solo livello**, e il mio albero ne ha sei. Le alternative erano mettere la traversata dentro `Section` — ma allora la struttura conoscerebbe anche la navigazione, due responsabilità in una classe — oppure esporre la lista interna, rompendo l'incapsulamento. La prova che la separazione serviva è che ne ho **due**, con ordini diversi, sullo stesso albero immutato.

8. « Che cos'è l'Exception Shielding e come l'hai implementato? »

È il principio per cui un'eccezione interna non attraversa mai il confine dell'applicazione. Tre livelli. Primo: una gerarchia di eccezioni di dominio, ognuna con **due messaggi**, `getDiagnostic()` per il log e `getUserMessage()` per lo schermo. Secondo: `SafeExecutor`, unico punto di cattura, che avvolge ogni comando. Terzo: due canali di uscita disgiunti, collegati da un codice di correlazione. I livelli intermedi non catturano mai: si cattura solo dove si può decidere qualcosa.

9. « Come proteggi l'applicazione da input malevolo? »

Su tre fronti. I percorsi passano da `PathSanitizer`, che risolve, **normalizza** e verifica che il risultato resti dentro la cartella base: `normalize` deve venire prima del confronto, altrimenti `manifests/../../etc/passwd` supererebbe il controllo. I comandi passano da `InputSanitizer`, che usa una **whitelist** e non una `blacklist`: una `blacklist` è completa solo quanto la tua immaginazione, una `whitelist` è completa per costruzione. E tutto il resto è coperto da `SafeExecutor`.

10. « Che alternativa avevi? Che compromesso hai fatto? »

Per il `Composite` avevo valutato `Object` più `instanceof`, e l'ho scartata perché il compilatore smette di aiutarti e ogni tipo nuovo aggiunge un ramo ovunque. Il compromesso che ho accettato è che il contratto `Content` è **più largo del minimo**: `getLanguages()` esiste su ogni contenuto e per un'immagine restituisce sempre un insieme vuoto. L'alternativa sarebbe stata spezzarlo in interfacce più piccole, ma avrei reintrodotta l'`instanceof` nel codice cliente. Ho scelto un contratto leggermente più largo in cambio di clienti che non fanno mai controlli di tipo.

Capitolo 6 — La demo, minuto per minuto

Sette comandi in ordine. Provala tre volte prima dell'esame.

```

1. load
  -> [OK] Manifest loaded - 218 contents, 55 languages
  Dì: "Ha appena letto 250 righe da un CSV e ne ha costruito un albero
      di sei livelli. La Factory ha deciso classe per classe."

2. tree
  -> l'albero indentato, con durata e conteggio per ogni nodo
  Dì: "Ogni riga di questa stampa e' una chiamata polimorfa:
      display() sulla sezione stampa se stessa e chiede ai figli."

3. budget
  -> Side 1  54:35 / 55:00  [OK]
      Side 2  60:28 / 55:00  [OVER]
      DISC   115:03 / 110:00 [OVER by 5:03]
  Dì: "Questi numeri sono la somma ricorsiva del Composite."

4. play ita
  -> [0:03] GREETING - Italian - LIVING
      "Tanti auguri e saluti."
  <-- QUESTO E' IL MOMENTO. Fermati un attimo.

5. validate
  -> [FAIL] Capacity - Side 2 exceeds its budget by 5:28
      Result: NOT ENGRAVABLE - 2 blocking issue(s)

6. curate coverage
  -> Policy: coverage
      Side 2 must free 5:28:
          - Liu Shui ("Flowing Streams"), guqin
  Dì: "Qui c'e' lo Strategy. Con 'curate count' cambia il criterio
      e la struttura del disco non se ne accorge."

7. load ../../etc/passwd
  -> [ERR-000006] The requested file path is not allowed.
  Dì: "Nessun crash, nessuno stack trace. Il dettaglio tecnico e'
      nel file curator.log, collegato da questo codice."
  E POI APRI curator.log. E' la prova.
```

6.1 · Le tre regole per il colloquio

1. **Mai dire « perché è una buona pratica ».** È la frase che segnala che hai imparato la conclusione senza il ragionamento. Struttura ogni risposta come **decisione → ragione tecnica → conseguenza concreta**.
2. **Quando non sai, dillo e circoscrivi.** « Questo non l'ho approfondito, ma il ragionamento che farei è... » vale molto più di un'invenzione.

3. **Mostra invece di raccontare.** Hai un terminale, 218 contenuti veri, un manifest deliberatamente fuori budget e due grep che chiudono la domanda sulla sicurezza. Ogni volta che puoi rispondere eseguendo qualcosa, fallo.

6.2 · Il conto

	Punti	Voci
α obbligatori	30 / 30	Composite 4 · Factory 3 · Iterator 4 · Exception Shielding 5 · Collections 3 · Generics 3 · I/O 3 · Logging 2 · JUnit 3
β opzionali	7 / 25	Strategy 4 · Builder 3 — scelti, non accumulati
γ penalità	0	Ogni voce ha una contromisura progettata
Totale	37 / 50	Approvazione a 25; il massimo a 50

Una scelta consapevole, da saper difendere

Con Observer, Chain of Responsibility, Memento e Template Method avresti saturato β e sfiorato i 50. Li abbiamo tolti su tua richiesta, ed è una posizione difendibile — ma va **detta**, non subito.

« Ho preferito un progetto che so difendere in ogni sua riga a uno più ricco che saprei difendere a metà. Un pattern è un costo pagato in indagine, e si paga quando il problema lo chiede. »

Se poi vorrai risalire, il candidato naturale è **Observer** su `Section.addContent()`: vale 5 punti, si aggiunge senza toccare nulla di esistente, e il professore non lo ha mai usato — quindi è anche il più difficile da difendere. Valuta tu.